

P1282R0

Ceci N'est Pas Une Pipe: Adding a workflow operator to C++

Published Proposal, 2018-09-27

This version:

<https://wg21.link/p1282r0>

Author:

[Isabella Muerte](#) (Target)

Audience:

EWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1282R0](#)

Current Source:

[slurps-mad-rips/papers/proposals/workflow-operator.bs](https://ericniebler.com/2018/09/27/slurps-mad-rips/papers/proposals/workflow-operator.bs)

Abstract

Adding a workflow operator to C++ gives us the opportunity to solve operator precedence for continuations on ranges, executors, monads, coroutines, and anything else users might want.

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design

4 FAQ

- 4.1 Does this have use beyond the stated examples so far?
- 4.2 Does this work like the `|>` operator from `F#`?
- 4.3 Are there any plans to add `<|>` to this set of operators?
- 4.4 Does this solve the same problem as UFCS?
- 4.5 Isn't this just UFCS?
- 4.6 I dunno, seems like UFCS to me
- 4.7 OK, it's just that it looks like its UFCS

5 Examples

- 5.1 Ranges
- 5.2 Monads
- 5.3 Coroutines And Executors
 - 5.3.1 Case 2
 - 5.3.2 Case 3
 - 5.3.3 Case 4

6 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial release. 🎉

§ 2. Motivation

With the advent of Ranges, Executors, Coroutines, and monadic interfaces upon us for C++20, we have a unique opportunity to create a uniform workflow interface for expanding on each of the operations provided by the aforementioned types.

This paper proposes two operators, hereby dubbed the "workflow operators", because they are not pipes, but for workflows. These workflow operators are represented with the characters `|>` and `<|`. If you are currently seeing two arrows on your screen, instead of a `|` and `>/<` combo, you probably are using a monospaced font with ligatures enabled. This paper does not recommend unicode specific characters.

§ 3. Design

The plan is to define these operators' precedence such that they are above the comma operator, but below the compound assignment operators. This should, ideally, allow them to begin to be written in code with minimal interference needed.

Currently, ranges and executors plan on overloading the `|` operator. This has unidirectional meaning, but more importantly has precedence in between the "bitwise xor" (`^`) and "logical and" (`and`) operators. This can cause odd and unexpected behavior. One could argue that overloading `>>` and `<<` was a mistake at the time. Perhaps we should take the opportunity to not repeat the same type of mistake.

Adding these operators has specific consequences, namely we don't need a paper everytime someone wants to add some kind of monadic operation to `std::optional`, `std::expected/outcome` or similar monadic types. In the past, people such as Ben Deane and Simon Brand, have spoken that the best operator we can get for proper binding is to use `>>`. However, its precedence means that we actually have to use `>>=`. Instead of overloading *that* operator, why don't we overload `|>` instead?

We also would now have an operator that binds correctly when chaining coroutines. Because `co_await` (or whatever possible glyphs the committee chooses) binds so tightly, we are currently unable to safely chain coroutines in continuations without having to rely on member functions, or passing said coroutines into another function to be executed later.

While it is asking *a lot* to add more operators into C++20, the author believes that this is an important requirement for composable and extensible interfaces regarding the aforementioned executors, ranges, coroutines, monads, and possibly even more interfaces.

§ 4. FAQ

§ 4.1. Does this have use beyond the stated examples so far?

Several people on the cplusplus slack channel have mentioned that it would be useful for function composition and binding, as well as a possible "infix" operator.

§ 4.2. Does this work like the `|>` operator from F#?

The syntax is taken from F#, however the semantics differ greatly due to F# being a garbage collected language, and an ML. C++ however is not either of those and more things must be taken into account.

§ 4.3. Are there any plans to add `<|>` to this set of operators?

At present there is no plan, however someone will probably find a use for it 😊

In all seriousness, there are possible uses for it *if* (and only if) C++ gets, perhaps, channels as found in languages such as Rust or go. That said, the current `|>` and `<|` do not remove the need for, say, `when_all`, and `when_any` on futures or future-like types.

§ 4.4. Does this solve the same problem as UFCS?

No. UFCS gets us one step closer to concept maps and uniform interfaces. This solves a different set of problems regarding operator precedence and extensible interfaces.

§ 4.5. Isn't this just UFCS?

No. UFCS has different semantics and requirements. This is just a regular old operator users and library implementors can utilize to create extensible interfaces with minimal interference into the way we write code.

§ 4.6. I dunno, seems like UFCS to me

It's not UFCS. We have a new operator in C++20 (`<=>`). We still don't have UFCS. This isn't UFCS. Stop saying it's UFCS. *It's not UFCS.*

§ 4.7. OK, it's just that it looks like its UFCS

No.

§ 5. Examples

The following examples show ranges, coroutines, and monad operations when chaining operations via the workflow operators.

§ 5.1. Ranges

The following is taken (and modified) from the original ranges v3 draft

```
int total = accumulate { 0 } <| view::iota(1)
               |> view::transform([](int x){return x*x;})
               |> view::take(10);
```

In the above we create a range on the righthand side of `<|` and then pass this in to our imaginary `accumulate` type that takes a range after being initialized with its initial value. This could also technically be written as

```
int total = view::iota(1)
               |> view::transform([](int x){return x*x;})
               |> view::take(10)
               |> accumulate(0);
```

§ 5.2. Monads

The following code was provided by Simon Brand via his CppCon 2018 talk on monads. It has been slightly modified to cut down on length. It is a demonstration of how we can extend existing interfaces without having to modify existing syntax or creating a new DSL when interacting with standard (or soon to be standard) types.

Note: This code is, minus the use of `operator |>` valid C++17 code. With the addition of Concepts, we could theoretically further constrain what operations or interfaces can be flowed into a monadic type.

```

template<typename T, typename E, typename L>
auto map(std::expected<T, E> const& ex, L const& fun)
-> expected<decltype(fun(*ex)), E> {
    if (ex) { return fun(*ex); }
    return std::make_unexpected(ex.error());
}

template<typename T, typename E, typename L>
expected<T, E> join (expected<T, E> const& ex, L const& fun) {
    if (ex) { return *ex; }
    return std::make_unexpected(ex.error());
}

template<typename T, typename E, typename L>
auto and_then( expected<T, E> const& ex, L const& fun) {
    return join(map(ex, fun));
}

namespace infix {

template<typename T>
struct map {
    T t;
    map(T const& t) : t(t) {}
};

template<typename T>
struct and_then {
    T t;
    and_then(T const& t) : t(t) {}
};

template<typename T, typename E, typename L>
auto operator|>(std::expected<T, E> const& ex, map const& fun) {
    return ::map(ex, fun.t);
}

template<typename T, typename E, typename L>
auto operator|>(std::expected<T, E> const& ex, and_then const& fun) {
    return ::and_then(ex, fun.t);
}

} /* namespace infix */

```

```

std::expected<int, string> divide (int numerator, int denominator) {
    if (denominator == 0) { return std::make_unexpected( "divide by zero" );
    return numerator / denominator;
}

std::expected<int, std::string> to_int(std::string const& s) {
    if (s.empty()) { return std::make_unexpected("string was empty"s); }
    int i;
    auto result = from_chars(&s.front(), &s.back(), i);
    if (result.ec or result.ptr != &s.back()) {
        return std::make_unexpected("string did not contain an int"s);
    }
    return i;
}

int main() {
    using namespace infix;
    auto result =
        to_int("12")
        |> and_then([] (int i) { return divide(42, i ); })
        |> map([] (double d) { return d * 2; });
}

```

§ 5.3. Coroutines And Executors

The following code is modified regarding coroutines and its (possible) interactions with executors found in [\[p1056r0\]](#). While this code may not seem very explanatory, the interaction between coroutines and executors is still not well defined. However, unlike the need to implement `.via` as a member function and having to "await" any changes if newer and better interfaces are found, users are free to implement their own interfaces on `task<T>`, coroutine types, executors, or a combination of them.

Recall, however, that a coroutine *starts* suspended and it is up to the user to decide how it is executed and where its continuation is placed.

§ 5.3.1. Case 2

```

co_await f() |> via(e);

```

§ 5.3.2. Case 3

Starts by an executor `ex`, resumes in the thread that triggered the completion of `f`

```
co_await spawn(ex) |> f;
```

§ 5.3.3. Case 4

Starts on executor `ex1`, resumes on executor `ex2`

```
co_await spawn(ex1) |> f |> via(ex2)
```

§ 6. Acknowledgements

Special thanks for Simon Brand for providing his source code. The encouragement of Simon Brand, Phil Nash, David Hollman, and countless others (I'm so sorry, there's a LOT of you and I can't remember all of you) to move forward with this paper after I originally suggested it in passing to Louis Dionne at CppCon

2017.

§ References

§ Informative References

[P1056R0]

Lewis Baker, Gor Nishanov. [Add coroutine task type](https://wg21.link/p1056r0). 5 May 2018. URL: <https://wg21.link/p1056r0>